RETAILPULSE – SECURE CLOUD DATA INGESTION & ALERTING PIPELINE

AWS Data Engineering Assignment Report

Submitted By: Parth Shah

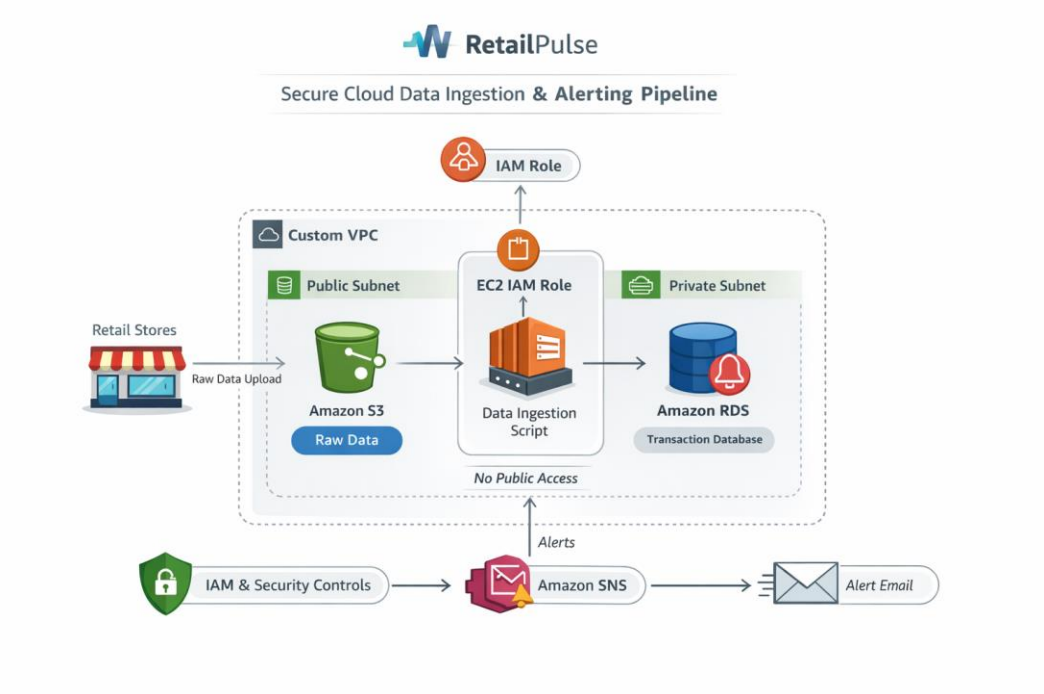
---

1. Introduction

RetailPulse is a retail analytics company that processes daily transaction data from multiple retail stores.
This project demonstrates the implementation of a secure and scalable AWS-based data pipeline for uploading, processing, storing, and monitoring transaction data.

The pipeline uses the following AWS services:

- **IAM** for access control
- **VPC** for network isolation
- **S3** for file storage
- **EC2** for script execution
- **RDS** for database storage
- **SNS** for failure alerts



---

2. Objective

The objective of this project is to build and manually execute a secure cloud data pipeline that performs the following tasks:

- Upload raw transaction files to **Amazon S3**
- Execute a Python ingestion script on **EC2**
- Validate and process transaction data
- Load processed records into **Amazon RDS**
- Send failure alerts through **Amazon SNS**

---

3. Architecture Overview

Pipeline Flow

1. Raw transaction CSV files are uploaded to **Amazon S3**
2. The ingestion script is manually executed on **EC2**
3. The script validates and processes the data
4. Processed records are inserted into **Amazon RDS**
5. On failure, an alert is sent via **Amazon SNS**

Security Measures

- IAM users, groups, and roles were used instead of root credentials
- S3 bucket public access was blocked
- RDS was deployed in a private subnet
- Security groups restricted DB access only to EC2
- VPC Flow Logs were enabled

---

4. IAM Setup

IAM users, groups, and policies were created following the principle of **least privilege**.

Implemented

- Created IAM users and groups
- Assigned users to groups
- Attached required policies
- Avoided use of root account after initial setup

Screenshot – IAM Users and Groups

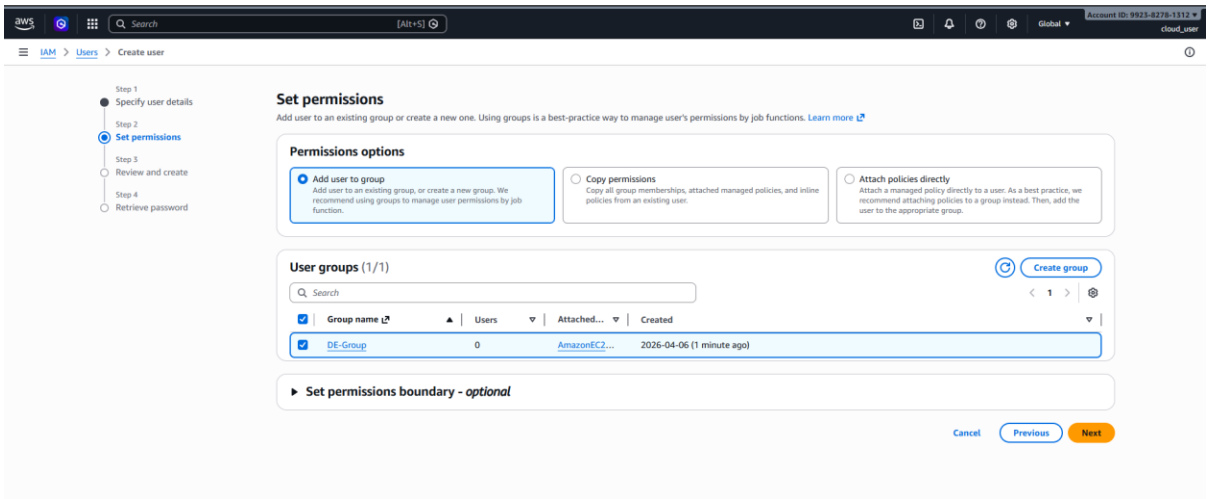


Figure 4.1: IAM users and groups configuration.

Screenshot – EC2 IAM Role Attachment

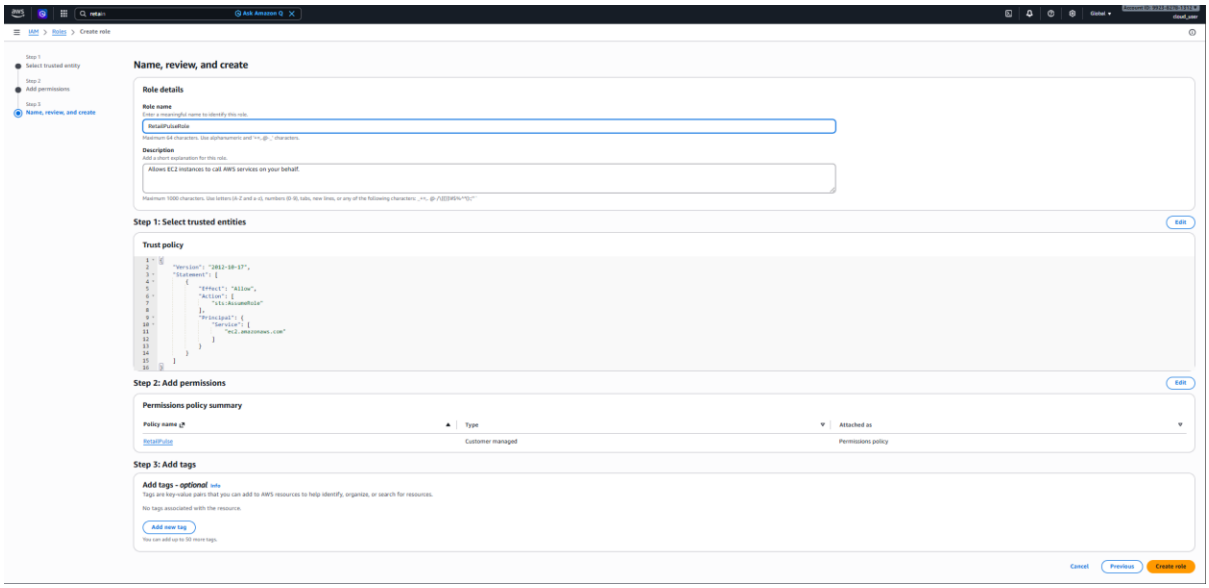


Figure 4.2: IAM role attached to EC2 instance.

---

5. VPC and Network Security

A custom VPC was created to securely deploy EC2 and RDS.

Implemented

- Created a **custom VPC**
- Created a **public subnet** for EC2
- Created a **private subnet** for RDS
- Attached an **Internet Gateway**
- Configured **route tables**
- Created **security groups**
- Enabled **VPC Flow Logs**

Security Rules

- EC2 was deployed in the public subnet
- RDS was deployed in the private subnet
- Database access was allowed **only from the EC2 security group**

Screenshot – VPC Overview

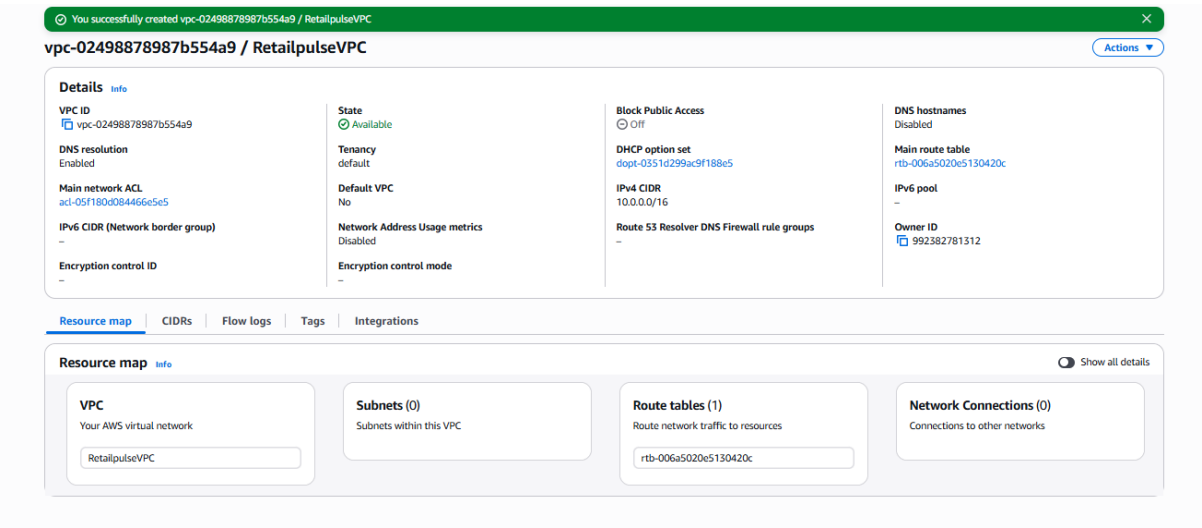


Figure 5.1: Custom VPC configuration.

Screenshot – Subnets

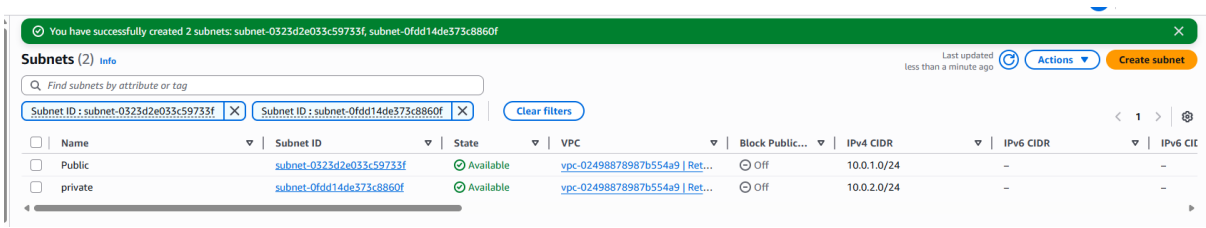


Figure 5.2: Public and private subnet configuration.

Screenshot – Security Group Rules

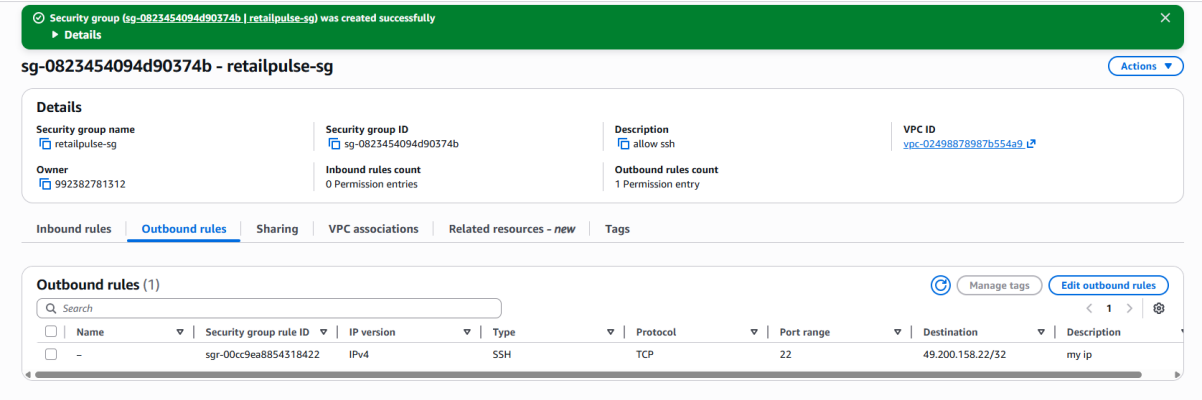


Figure 5.3: Security group rules for EC2 and RDS.

---

6. Amazon S3 Setup

Amazon S3 was used to store raw and processed transaction data securely.

Implemented

- Created a unique S3 bucket
- Blocked all public access
- Created folder structure:

raw/pos/
raw/ecommerce/
processed/pos/
processed/ecommerce/

- Applied restricted access policy for EC2 IAM role only
- Configured lifecycle rules:
  - `raw/pos/` → Expire after 30 days
  - `raw/ecommerce/` → Expire after 30 days

Screenshot – S3 Bucket Permissions

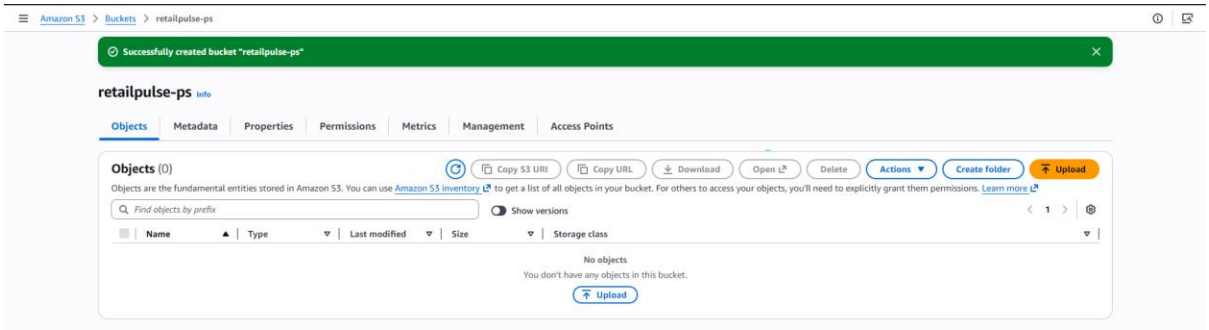
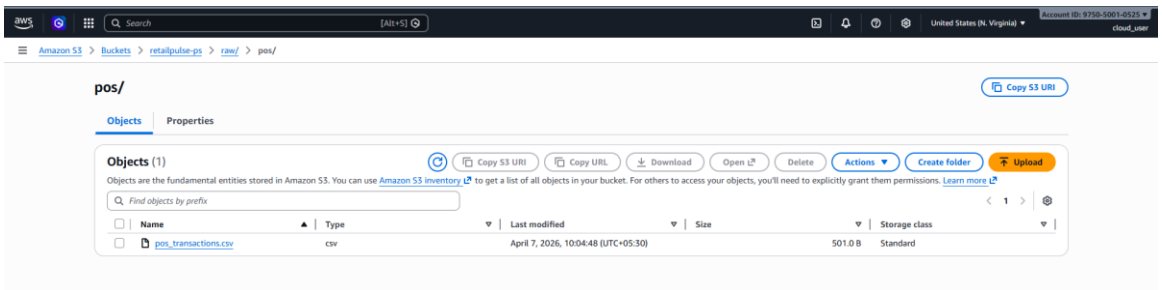


Figure 6.1: S3 bucket public access blocked.

Screenshot – Files in raw/pos/

Insert Screenshot Here



Screenshot – Files in raw/ecommerce/

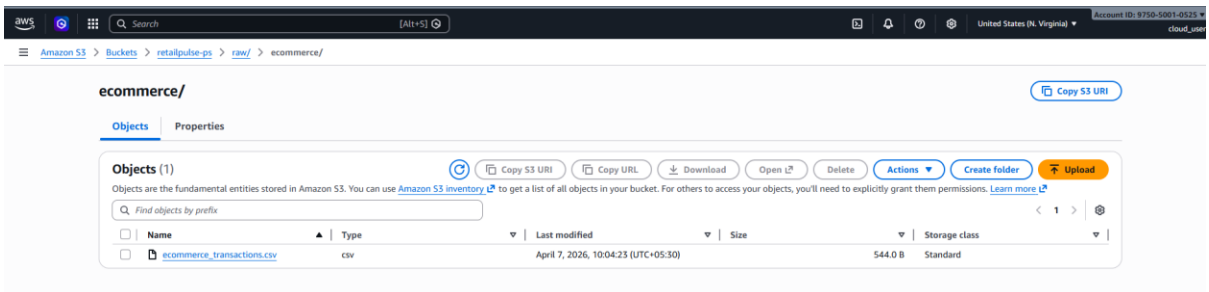


Figure 6.3: Raw ecommerce transaction files uploaded to S3.

---

7. EC2 – Manual Data Ingestion Pipeline

An EC2 instance was launched to manually execute the ingestion pipeline.

Installed on EC2

- Python 3
- AWS CLI

- Required Python libraries (`boto3`, `pandas`, DB connector)

Screenshot – EC2 Instance Details

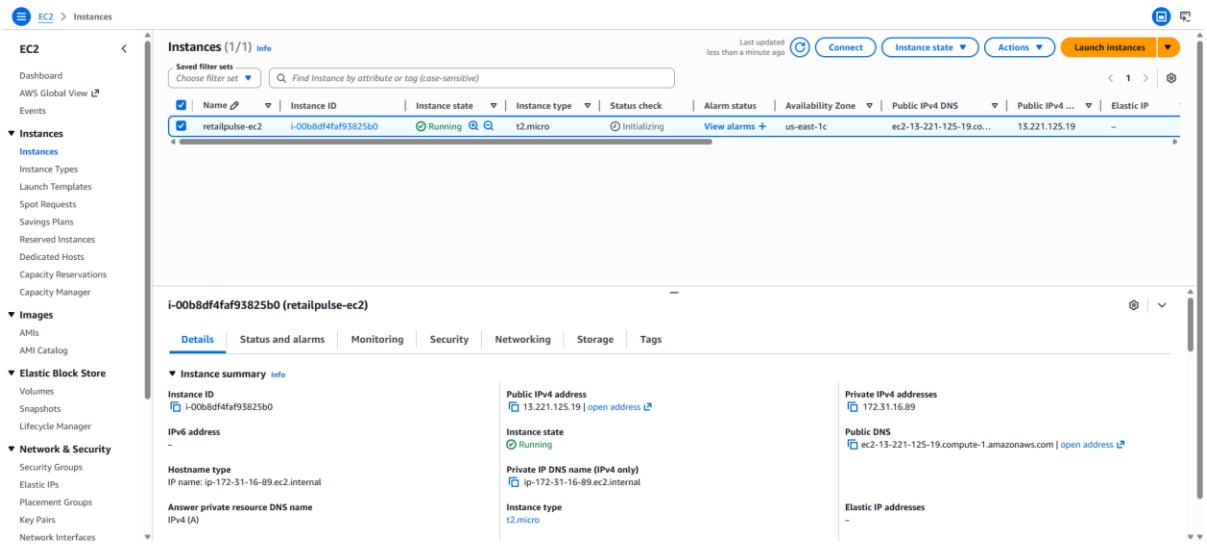


Figure 7.1: EC2 instance configuration.

Screenshot – Manual Script Execution

```python
import boto3
import pandas as pd
import psycopg2

S3_BUCKET = "retailpulse-ps"
SNS_TOPIC_ARN = "RetailAlerts"

s3 = boto3.client('s3')
sns = boto3.client('sns')

def alert(msg):
    sns.publish(TopicArn=SNS_TOPIC_ARN, Message=msg)

try:
    df = pd.read_csv("pos_transactions.csv")

    if df.empty:
        raise Exception("Empty file")

    s3.upload_file("pos_transactions.csv", S3_BUCKET, "raw/pos/pos.csv")

    df.dropna(inplace=True)
    df.to_csv("processed.csv", index=False)

    s3.upload_file("processed.csv", S3_BUCKET, "processed/processed.csv")

    conn = psycopg2.connect(
        host="retail.cpyaea8kqp86.us-east-1.rds.amazonaws.com",
        database="retail",
        user="admin1",
        password="password123"
    )

    cur = conn.cursor()

    for _, row in df.iterrows():
        cur.execute("INSERT INTO transactions VALUES (%s,%s,%s,%s)", tuple(row))

    conn.commit()
    cur.close()
    conn.close()

    print("SUCCESS")

except Exception as e:
    alert(str(e))
    print("FAILED:", e)
```

Figure 7.2: Manual execution of ingestion script from EC2 terminal.

Execution Command

python3 script.py

8. Amazon RDS Setup

Amazon RDS was used to store processed transaction records.

Implemented

- Created an RDS instance (**MySQL / PostgreSQL**)
- Deployed the database in a **private subnet**
- Disabled public access
- Connected to RDS from EC2
- Created tables and inserted processed records

Screenshot – RDS Instance Configuration

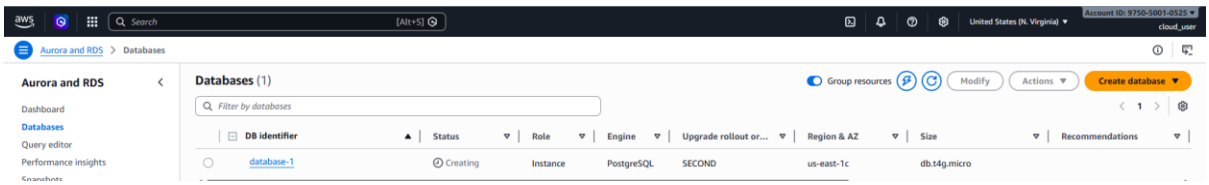


Figure 8.1: RDS instance deployed in private subnet.

Database Connection from EC2

```
psql -h database-1.crqoqa0wysyu.us-east-1.rds.amazonaws.com -U admin1 -d retail
```

Example Query

SELECT * FROM transactions LIMIT 10;

---

9. Amazon SNS – Alerts and Monitoring

Amazon SNS was configured to send email alerts for pipeline failures.

Implemented

- Created an SNS topic
- Subscribed using email
- Modified the script to publish alerts when:
  - Script execution fails
  - Database insertion fails

A failure was manually triggered to validate alert delivery.

Screenshot – SNS Topic and Subscription

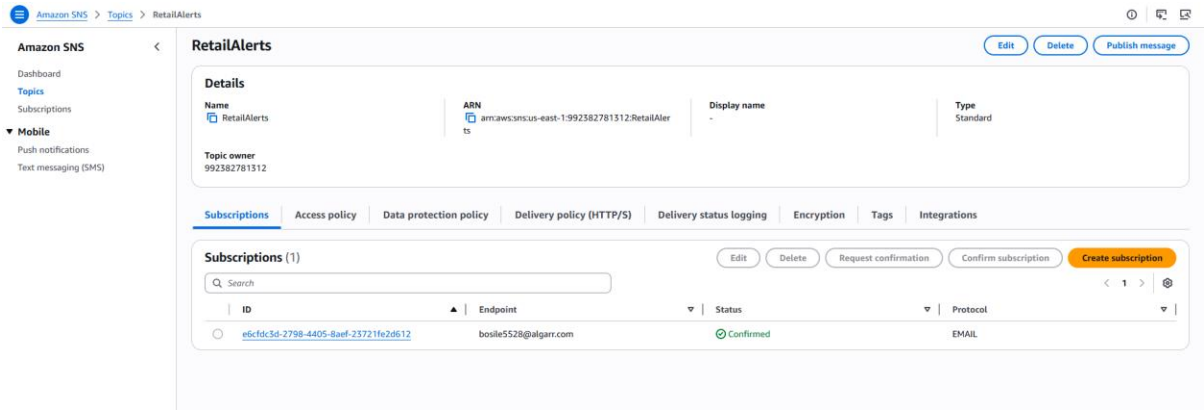


Figure 9.1: SNS topic and email subscription.

Screenshot – Received Alert Email

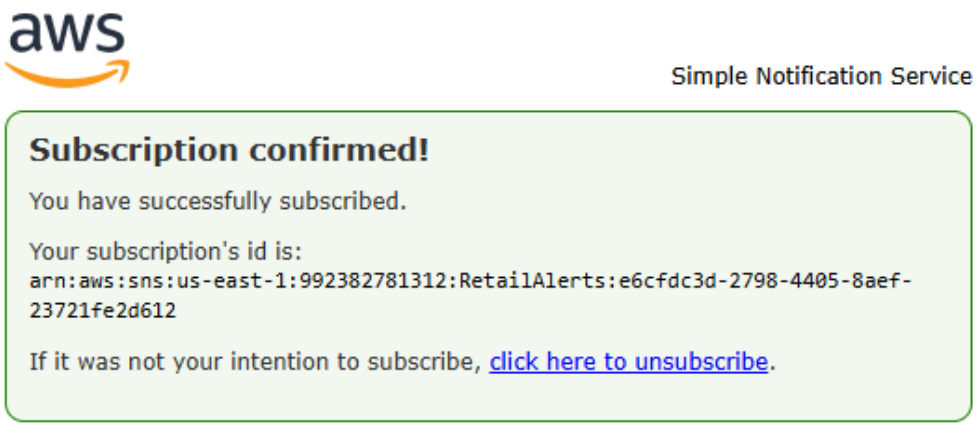


Figure 9.2: Email alert received from Amazon SNS.

---

10. Pipeline Validation

The complete pipeline was manually executed and validated successfully.

Validated Results

- Raw transaction files uploaded to **S3**
- Python ingestion script executed on **EC2**
- Records inserted into **RDS**
- Failure alerts received via **SNS**
- Security best practices followed throughout the implementation

11. Conclusion

This project successfully implemented a secure AWS-based retail transaction ingestion pipeline for RetailPulse.

The solution demonstrates practical use of AWS services for:

- Secure access control
- Cloud storage
- Compute-based ingestion
- Relational database loading
- Monitoring and alerting

The final pipeline follows real-world enterprise cloud engineering practices and satisfies all assignment requirements.